

5.2 Team AI Usage

Iliyas Naizabek — Member 1, Firewall Core and High Availability

AI was used during this project as an explainer, code reviewer, and debugging tool. It was not used to generate complete solutions without review. Every AI output was read and understood before being accepted, tested by running the actual commands, and corrected when the output did not match the real project environment or contained bugs.

The three prompt moments below represent the most significant uses of AI in my contribution: understanding the firewall concept, fixing real bugs in Python and Jinja2, and solving the Docker networking constraint for Keepalived.

Prompt Moment 1 — Understanding nftables and Designing the Firewall Structure

Why this prompt moment was selected

This moment shows that I did not start by asking AI to write firewall rules immediately. I first shared the project requirements and asked AI to explain how nftables works, what stateful filtering means, and how to structure a correct firewall with a default-drop policy. This reflects responsible AI use: understand the concept first, then build.

Prompt excerpt

After reading the project description I asked:

Explain how nftables works for a stateful firewall. What is the difference between a stateful and stateless firewall? What does policy drop mean and why is it important for a secure firewall? I need to implement this on Debian 13 for a high-availability cluster.

I then followed up with:

Now write the base nftables configuration with: table inet filter, input chain with policy drop, rules for established/related connections, loopback, ICMP and ICMPv6, SSH port 22, HTTP port 80, HTTPS port 443. Explain each rule.

Manual review and reflection

The AI explanation helped me understand why `ct state established,related accept` must come before any port-specific rules — otherwise reply packets to already established connections would be dropped. This was not obvious from reading the nftables man page alone.

The AI output also used table inet filter which handles both IPv4 and IPv6 in one table. I verified this was correct for our Debian 13 target by testing it with `nft -c -f` syntax check before applying.

I also caught one issue: the AI initially used **policy accept** as an example default. I corrected this immediately because the project requirement is a secure firewall, not a permissive one. The final config uses **policy drop** on all chains.

Project result

This prompt chain produced the base `nftables.conf` and informed the structure of both Jinja2 templates. The resulting firewall was verified with:

```
sudo nft list ruleset
```

Output confirmed policy drop on input, correct ct state handling, and all required accept rules:

```
chain input {
    type filter hook input priority filter; policy drop;
    ct state established,related accept
    iif "lo" accept
    tcp dport 22 accept
    tcp dport 80 accept
    tcp dport 443 accept
    ip protocol icmp accept
}
```

Prompt Moment 2 — Debugging the Jinja2 Template and Python `render.py`

Why this prompt moment was selected

This moment shows how AI was used to find and fix real bugs in Python code and Jinja2 templates. The AI did not write the code from scratch — it reviewed broken code I had already written and identified specific errors. This is a good example of AI as a debugging tool, not just a code generator.

Prompt excerpt

After writing `render.py` I ran it and got errors. I shared the broken code with the AI:

```
Here is my render.py. It crashes or produces wrong output. Find all bugs and explain each one:
with open('vars.yml') as f:
    data = yaml.safe_load(f)
resolved = []
for hostname in vars_data.get('hosts', []):
    ... except socket.gaierror as e:
print(f"WARNING: {hostname} can't be resolved. Error: {e}")
resolveeg.append(entry)
vars_data['resolved_hosts'] = resolved
```

I verified each fix by running the corrected script and checking the output. The hostname resolution was then tested with real hostnames (`localhost`, `debian`) and the output confirmed correct IPv4 resolution:

```
ip saddr 127.0.0.1 accept comment "localhost"
ip saddr 127.0.1.1 accept comment "debian"
```

The AI also caught that the same `resolved_hosts` block appeared twice in the Jinja2 template, which caused every rule to appear duplicated in the generated file. After removing the duplicate block and adding `flush ruleset` at the top of the template, the output was clean.

Project result

This prompt chain fixed `render.py`, cleaned `templates/nftables_ipv4.j2`, and produced correct `generated_ipv4.nft` and `generated_ipv6.nft` files. Both files pass `nft -c -f` syntax validation and apply without errors.

Prompt Moment 3 — Keepalived Unicast for Docker and Custom Ansible Module

Why this prompt moment was selected

This moment covers two related decisions that required understanding Docker networking constraints before writing any configuration. AI helped explain why standard Keepalived VRRP multicast does not work in Docker and what unicast configuration is needed instead. It also guided the design of the custom Ansible module for idempotent nftables rule management.

Prompt excerpt

After installing Keepalived inside the Docker containers I asked:

I am trying to run Keepalived inside Docker containers. Standard VRRP uses multicast but it is not working. Why does multicast not work in Docker bridge networks? What is the correct Keepalived configuration to use unicast instead? I have two containers: `node1` at `172.20.0.2` and `node2` at `172.20.0.3`. The VIP should be `172.20.0.100`.

After Keepalived was working I asked:

Write a custom Ansible module in Python called `nftables_rule` that can add or remove a single nftables rule. It must be idempotent: if the rule already exists, running it again must return `changed: false`. If the rule does not exist, it adds it and returns `changed: true`. Use `nft list chain` to check existence and `nft add rule` to add. Support `state: present` and `state: absent`.

Manual review and reflection

The AI explanation of Docker multicast was accurate: Docker bridge networks do not forward VRRP multicast packets between containers because the bridge does not pass layer 2 multicast by default. Unicast mode solves this by sending VRRP advertisements directly to peer IP addresses.

I verified the unicast configuration worked by checking VIP assignment after starting Keepalived:

```
sudo docker exec node1 ip addr show eth0 | grep 172.20.0.100
```

Output confirmed:

```
inet 172.20.0.100/24 scope global secondary proto keepalived eth0
```

For the Ansible module, the AI first produced a version that used `subprocess.run` without `capture_output=True`, which meant output was not captured and the idempotence check could not work correctly. I caught this by reading the code before running it and asked for a correction.

The corrected module was then tested with two consecutive playbook runs:

Run	Result	Meaning
First run	changed=1	Rule was added to nftables
Second run	changed=0	Rule already existed, no change made

This confirmed correct idempotent behavior, which is the core requirement for any Ansible module.

The failover test was also guided by AI: stop Keepalived on node1, wait 4 seconds, check VIP on node2. The AI suggested using a loop with sleep intervals rather than a fixed wait, which I implemented in `test_failover.sh`.

Project result

This prompt chain produced:

```
keepalived/node1.conf and keepalived/node2.conf — unicast VRRP
configurationlibrary/nftables_rule.py          — idempotent custom Ansible
moduletest_failover.sh                         — three-scenario failover
testcheck_sync.sh                             — conntrackd sync verification
```

Final test result:

```
=====
FAIL=0===== Results: PASS=3
ALL TESTS PASSED
```

Summary

AI was used throughout this project as an explainer, code reviewer, and debugging tool. It was not used to generate complete solutions without review. Every AI output was read and understood before being accepted, tested by running the actual commands, and corrected when the output did not match the real project environment or contained bugs.

The three prompt moments above represent the most significant uses of AI in my contribution: understanding the firewall concept, fixing real bugs in Python and Jinja2, and solving the Docker networking constraint for Keepalived. All other work — running commands, reading output, making decisions about configuration — was done manually.

Said Darkhanuly - Member 2, Testing Environment + Docker + Tests

Prompt Moment 1 - Architecture Understanding Before Coding

Why this prompt moment was selected

This moment shows that I did not start by asking AI to write code immediately. I first shared the full project zip and asked AI to analyze what already existed and explain the full architecture before writing anything.

Prompt excerpt

I shared the project zip and asked AI to explain the topology, identify and suggest the right order to implement my tasks. AI produced an architecture diagram showing all containers, all three networks, and traffic flow before any code was written.

Manual correction after AI output

The analysis revealed three concrete bugs in the existing files: wrong Dockerfile name in docker-compose.yml, entrypoint.sh referenced at the wrong path, and keepalived and conntrackd configs using IPs that did not match the compose network assignments. I verified each one manually before accepting the corrections.

Project result

This prompt chain set the working method for the entire testing contribution and produced the file placement map that defined where each of the 11 deliverables belongs in the project structure.

Prompt Moment 2 - Docker Compose Topology With Three Isolated Networks

Why this prompt moment was selected

This example shows how the Docker Compose file and the three isolated networks were designed with specific constraints from the team architecture.

Prompt excerpt

After the architecture was clear I asked AI to write the complete docker-compose.yml with 3 firewall nodes, 3 client nodes, isolated networks, static IPs, health checks, and a test runner that only starts when explicitly called. I constrained the answer with the exact subnet plan: mgmt_net for Keepalived and Ansible, frontend_net for client traffic, and backend_net for protected servers.

Manual correction after AI output

I verified the static IP addresses against the keepalived and conntrackd config files to confirm they matched across all files. I also checked that net.ipv4.ip_nonlocal_bind was present in sysctls, because without it Keepalived cannot bind the Virtual IP before it is assigned to the interface.

Project result

This prompt chain produced the final docker-compose.yml with three networks, six application containers, two backend servers, and a dedicated test runner. The static IP map became the reference used by all test files, the health monitor, and the Ansible inventory.

Prompt Moment 3 - Failover Tests With Fixture-Based Cleanup

Why this prompt moment was selected

This example shows a concrete guided coding correction. AI first produced a working test but with unsafe cleanup. The follow-up prompt and manual implementation corrected it by referencing the correct pytest pattern for test isolation.

Prompt excerpt

I asked AI to help me write failover tests that stop fw1 with docker stop, poll for VIP ownership by checking ip addr show inside containers, assert fw2 wins due to higher priority, and always clean up stopped containers even if an assertion fails. I specified that cleanup must use a pytest yield fixture, not a try/finally block inside the test itself.

Manual correction after AI output

The first output placed cleanup inside a try/finally block directly in the test function. I corrected this by correcting yield fixture in conftest.py. The corrected version uses restore_fw1 as a function-scoped fixture that always runs teardown after each test, which is the correct pytest pattern for test isolation.

Project result

This prompt chain produced tests/test_failover.py and the shared helper functions in tests/conftest.py, covering initial MASTER election, single node failure, priority-based backup selection, VIP reachability after failover, preemption on recovery, and double failure scenarios.

Aisana Sembek – Member 4, Monitoring and Diagnostics

AI was used during this project as an explainer, code reviewer, and debugging partner for the monitoring component (drop logging, blocked-IP analysis, connection tracking, dashboard, bandwidth visibility, rule integrity, and Telegram alerts). It was not used to generate complete solutions without review. Every AI output was applied to the live Docker stack and verified (nftables counters, log files, HTTP endpoints, Telegram messages) before being accepted, and corrected when it did not match the real project environment or contained bugs.

The three prompt moments below represent the most significant uses of AI in my contribution: making firewall drops observable inside Docker, fixing the connection analyzer against real output, and building the network bandwidth feature requested in the review.

Prompt Moment 1 – Making nftables Drop Logs Visible Inside Docker

Why this prompt moment was selected

This was the core problem of my part. The firewall drop logs were not visible inside the containers, and the first conclusion was that monitoring could only work on a real VM. This moment shows I used AI to understand the cause first, then implement and test a fix inside Docker, rather than accepting the limitation.

Prompt excerpt

I described the exact symptom and environment:

My nftables log rule works but nothing appears in the container log file. rsyslog is running and the log rule is added, but the container does not receive kernel messages. Why, and how can I make firewall drop logging work inside Docker?

Manual review and reflection

The AI explained that the standard nftables log statement writes to the kernel log, which is not namespaced per container, so a container cannot see it. It proposed switching to NFLOG (log ... group 1) together with a ulogd2 service that listens on the NFLOG group and writes the drops to a file.

I did not accept this on trust. I applied it on the fw1 container, generated blocked traffic from a client, and confirmed real log lines appeared with real SRC= addresses. I also caught an important follow-up problem myself: after a container restart the manual rule was lost and logging stopped. This showed the fix had to live in the code (entrypoint, Dockerfile, supervisord), not be applied by hand. A second issue was that the nftables package ships an empty /etc/nftables.conf, so the "write default only if missing" condition never triggered; I changed it to always write the ruleset and re-verified after a clean rebuild.

Project result

This produced the working drop-logging pipeline: the NFLOG rule in the ruleset, ulogd2 installed in the firewall image and started by supervisord, and drops written to /var/log/firewall/dropped.log. Verified with:

```
docker exec fw1 nft list ruleset | grep group docker exec fw1 cat  
/var/log/firewall/dropped.log
```

Output confirmed the rule (log prefix "DROP-IN " group 1 counter drop) and real log lines with live source IPs, which the log analyzer then counts.

Prompt Moment 2 – Fixing the Connection Analyzer Against Real conntrack Output

Why this prompt moment was selected

This moment shows AI used as a debugging tool on code that returned wrong results, and a case where I caught the real cause through testing rather than trusting the first explanation.

Prompt excerpt

After the dashboard connections endpoint kept returning zero, I asked:

My conntrack analyzer always returns total: 0, but the connection table is not empty. It calls conntrack -L -o json. Why could this return nothing, and how should I read the connections instead?

Manual review and reflection

By comparing the API output with the raw conntrack -L output in the container, I found the analyzer was returning zero even though real connections existed. The cause was that the -o json mode is not supported in our build, so the JSON parse silently failed. With AI help we rewrote the analyzer to parse the normal text output with a regular expression, grouping by protocol, state, and destination port.

I verified the fix on the live table: after the rewrite the endpoint returned real connections, for example {"by_protocol":{"tcp":1},"by_state":{"ESTABLISHED":1},"total":1}. I also confirmed that when the table is genuinely empty the analyzer returns real zeros, not demo data, which was a separate bug I had already removed earlier.

Project result

This fixed monitoring/analyzers/conntrack_analyzer.py so the dashboard shows real active connections grouped by protocol, state, and destination port, instead of always zero.

Prompt Moment 3 – Network Bandwidth Feature Requested in the Review

Why this prompt moment was selected

In the review the professor asked me to add per-network bandwidth to the dashboard, similar to what iftop and cbm show, and suggested finding out how those tools read the data from the Linux kernel. This moment shows using AI to learn the mechanism and then implement it myself.

Prompt excerpt

The professor asked me to show network bandwidth on my dashboard, like iftop or cbm. How do those tools get bandwidth data from the Linux kernel, and how can I read the same data with my own Python code?

Manual review and reflection

The AI explained that these tools read the per-interface byte counters exposed by the kernel, and that the simplest source in Python is /proc/net/dev: take two samples one

second apart and the difference is the throughput in bytes per second. This needs no extra libraries.

I implemented `bandwidth_analyzer.py` on this idea, added an `/api/bandwidth` endpoint, and rewrote the dashboard page to show live bars per network instead of raw JSON, which also addressed the professor's second suggestion about better visualization. I labelled the interfaces by network (Frontend, Backend, Management). I verified it under a real flood ping and saw the correct interface report several MB/s while idle interfaces stayed at zero.

Project result

This produced `monitoring/analyzers/bandwidth_analyzer.py`, the `/api/bandwidth` endpoint, and the updated dashboard with live bandwidth bars. To make the dashboard show live data automatically on startup, I also added a traffic-gen container to `docker-compose.yml`, and set up `alert_monitor.py` to send Telegram alerts for heavily blocked IPs (running on the host, because the firewall containers have no internet access, with the bot token passed via environment variables and never committed to git).

Summary

AI was used throughout my part as an explainer, code reviewer, and debugging partner. It was not used to generate complete solutions without review. Every AI output was applied to the live Docker stack and verified before being accepted, and corrected when it did not match the real environment or contained bugs. The most valuable moments were the ones where the AI's assumption was wrong, the conntrack JSON mode, the lost-on-restart ruleset, and the empty nftables stub and I found them through testing. All other work (running commands, reading output, and deciding what to keep) was done manually.

Shahzod Mukhamedzhanov – Member 3, High Availability, CI/CD and Rules for firewalls

AI was used during this project as an explainer, code reviewer, and debugging partner for the High Availability and CI/CD layer of my part (Keepalived/VRRP roles, the Ansible playbooks, the Docker Compose setup, and the Gitea CI/CD pipeline). It was not used to generate complete solutions without review. Every AI output was applied to the live Docker stack and verified (container status, nft/ip addr output, VRRP state, CI pipeline runs) before being accepted, and corrected when it did not match the real environment.

The three prompt moments below represent the most significant uses of AI in my contribution: building the Keepalived/VRRP Ansible playbooks, fixing the Gitea CI/CD pipeline, and resolving a Docker Compose networking problem.

Prompt Moment 1 – Keepalived/VRRP Ansible Playbooks

Why this prompt moment was selected

This was the core of my part: the cluster needed independent failover across three separate networks (mgmt, frontend, backend), not a single shared VIP. This moment shows I used AI to understand how to structure multiple VRRP instances correctly, then found and fixed a real split-brain bug myself through testing, rather than accepting the first working version.

Prompt excerpt

I need an Ansible role that configures Keepalived with three independent VRRP instances on three different networks (mgmt, frontend, backend), each with its own VIP and priority, so failover on one network doesn't affect the others. What's the correct way to structure the keepalived.conf.j2 template and role variables for this?

Manual review and reflection

The AI proposed a role structure with one vrrp_instance block per network, templated from role variables (interface, virtual router ID, priority, VIP, auth). I didn't accept this as final — I deployed it across all three firewall nodes and tested actual failover by stopping the master container on each network in turn. During this testing I found a problem the AI's version didn't account for: under certain restart orderings, two nodes both entered MASTER state on the same VRRP instance at once (split-brain), because the virtual router IDs and priorities overlapped between instances. I traced this by comparing ip addr show output across all three nodes during a forced restart sequence, and corrected the VRRP instance definitions so router IDs are unique per instance and priorities are strictly ordered per network.

Project result

This produced the keepalived role deploying three independent VRRP instances with no split-brain, verified with:

```
docker exec fw1 ip addr show | grep 172.20.0.100
```

```
docker exec fw2 ip addr show | grep 172.20.0.100
```

confirming only one node holds each VIP at a time, and that mastership moves correctly on failure.

Prompt Moment 2 – Gitea CI/CD Pipeline (run_tests.yml)

Why this prompt moment was selected

This moment shows AI used as a debugging tool on a pipeline that failed inconsistently, and a case where I found the real cause through testing rather than trusting the first explanation.

Prompt excerpt

My Gitea CI pipeline runs pytest, but it fails with "file not found" errors only sometimes, depending on what triggered the run. The tests exist and run fine locally. What could cause pytest to look in the wrong place inside a CI runner?

Manual review and reflection

The AI explained that pytest resolves paths relative to the current working directory the process is started from, and that CI runners don't always start in the repository root the way a local terminal session does, so relative paths in the workflow can point to different places depending on the trigger. I didn't accept this as a guess — I added debug steps to the workflow to print the actual working directory on both a manual run and a push-triggered run, confirmed they differed, and fixed `run_tests.yml` to set the working directory explicitly rather than relying on the runner's default.

Project result

This fixed `run_tests.yml` so the pipeline runs and passes consistently regardless of what triggered it, verified across multiple manual and push-triggered runs, with the full suite passing at 85 passed, 0 failed.

Prompt Moment 3 – Docker Compose Networking

Why this prompt moment was selected

This moment shows AI used to diagnose an intermittent infrastructure problem that was hard to reproduce, and a case where I verified the fix directly on the running stack rather than assuming the explanation was correct.

Prompt excerpt

Containers on my Docker Compose network sometimes can't resolve each other by container name, and it's not consistent — it works, then a few minutes later a lookup fails. What would cause intermittent DNS resolution failures between containers on the same Compose network?

Manual review and reflection

The AI pointed to Docker's embedded DNS server and suggested checking the network configuration in `docker-compose.yml` — specifically whether all the firewall containers were reliably attached to the same user-defined network with predictable service names, since intermittent resolution issues often come from containers

restarting into new IPs faster than DNS cache expiry, or from being attached to more than one network in a way that creates ambiguous resolution order. I tested this by restarting individual containers repeatedly while running continuous ping/getent hosts checks from another container, and confirmed the failures correlated with restart timing rather than being random. I corrected the network definitions in `docker-compose.yml` so name resolution became reliable across repeated restarts.

Project result

This fixed the Docker Compose network configuration so conntrackd replication and inter-node health checks no longer fail intermittently, verified by running repeated restart cycles with continuous name-resolution checks between all three firewall nodes.

Summary

AI was used throughout my part as an explainer, code reviewer, and debugging partner for the Keepalived/VRRP playbooks, the Gitea CI/CD pipeline, and the Docker Compose networking setup. It was not used to generate complete solutions without review. Every AI output was applied to the live Docker stack and verified (VIP ownership, VRRP state, CI pipeline runs, DNS resolution) before being accepted, and corrected when it did not match the real environment. The most valuable moments were the ones where I found the real cause myself through testing — the VRRP split-brain conflict, the CI working-directory inconsistency, and the Docker DNS timing issue — rather than accepting the AI's first explanation. All other work (running commands, reading output, and deciding what to keep) was done manually.

Yedige Mussabayev - Member 5, Ansible Integration

Three Selected AI Prompt Moments For The Group AI-Use Documentation

This section is a shortened and cleaned extract from my longer AI documentation for the Ansible part of the Cloud Computing project. It is meant for the common team document where each member contributes three representative prompts or prompt chains.

The selected examples are not only "code generation" prompts. They show the way AI was used: with course material, the professor's Ansible/Docker example, the agreed team architecture, existing project files, and real runtime checks as constraints. AI

output was treated as draft material and was accepted only after manual review and testing.

Prompt Moment 1 - Guided Coding Concept And Course Reference

Why this prompt moment was selected

This moment explains the general method behind my Ansible work. It connects the project to the course lecture on Ansible and to the professor's GitLab Ansible/Docker example. It also shows that the prompting process was guided and checked, not just automatic code generation.

Prompt and documentation excerpt

The Ansible part of the project was guided by the course lecture Cloud Computing 07 Ansible for desired state configuration. The lecture presented Ansible as a practical Infrastructure as Code tool: inventory and variables describe the desired state, and playbooks and templates render that state into concrete configuration files on managed hosts.

The professor's GitLab Ansible/Docker example was used as a conceptual reference for the workflow: define hosts, define variables, run playbooks, render configuration, and validate the behavior after rollout. Our project extended this from a simple Ansible host configuration example into a high-availability firewall laboratory.

My prompting concept was to start from the architecture agreed with teammates: a Docker Compose lab, three firewall nodes, client and backend networks, HA firewall behavior, and automated tests. I asked AI to help draft and review code, but always constrained the answers with the current project structure, file names, variable names, Docker container names, Ansible roles, and expected runtime behavior.

I rejected or corrected AI output when it invented new files, changed the variable contract, followed SSH assumptions that did not fit our Docker Compose lab, produced invalid Python/Jinja/nftables syntax, or looked plausible but did not match our working deployment.

Manual review and reflection

This prompt chain documents the responsible AI method used in the Ansible part. AI was useful for structure, alternatives, and first drafts, but the final decisions were based on the course concept, the professor's example, teammate architecture, and runtime validation.

The important verification steps were Ansible inventory graph, syntax check, ansible-lint, Docker container health, nft list ruleset, Keepalived VIP ownership, conntrackd status, and pytest integration tests.

Project result

This became the working method for the Ansible contribution. It explains why the final project uses inventory and group variables as the readable desired-state source, while playbooks and templates apply that state to the running Docker firewall lab.

Prompt Moment 2 - Inventory Design Based On The Ansible Lecture Example

Why this prompt moment was selected

This example directly reflects on the Ansible lecture and adapts its simple inventory idea to our more complex Docker Compose firewall lab. It is a good example of using the course material as a starting point while still correcting the AI output to match the real project.

Prompt excerpt

I want to improve the Ansible inventory file based on the Ansible lecture example from the course. In the lecture example the inventory was simple. It had a group of Debian hosts like host1, host2, and host3 with some host variables. The point of that example was to show that Ansible needs a clear inventory before it can apply desired state configuration.

In our project the same idea is used, but the environment is more complex because we have a Docker Compose firewall lab. Please help me write or review ansible/inventory/hosts.yml.

The inventory should describe the firewall cluster with fw1, fw2, and fw3 in the firewalls group and the child group firewall_cluster_main. It should also define backend servers and test clients in separate groups. Keep the firewall node variables that are really node-specific, such as addresses, inventory names, Keepalived state or priority, and conntrackd peer information.

Do not put full firewall rules into hosts.yml. Those belong in group variables, because firewall rules are shared desired-state data, not individual host identity. Do not include SSH key files or known_hosts handling in the inventory.

After that, explain how this inventory follows the idea from the lecture and why this structured inventory is better than a flat list of containers.

Manual correction after AI output

The first AI output followed the lecture's SSH-based example too closely. That was useful as a conceptual starting point, but it did not fit our runtime. Our project uses Docker Compose containers and the `community.docker.docker` Ansible connection plugin, so the corrected inventory uses container names such as `fw1`, `fw2`, `fw3`, `server1`, `server2`, `client1`, `client2`, and `client3`.

The final correction was: keep the lecture idea of a clear environment map, but adapt the connection mode and host identities to the actual Docker lab. This avoided unreliable SSH assumptions and made the Ansible deployment repeatable after containers are rebuilt.

Project result

This prompt chain affected `ansible/inventory/hosts.yml`. The resulting inventory separates firewall nodes, backend servers, and test clients, while shared policy and network objects stay in `group_vars`. This makes the deployment easier to review and closer to the desired-state approach taught in the course.

Prompt Moment 3 - Correcting `site.yml` Through Manual Review

Why this prompt moment was selected

This example shows a concrete guided coding correction. AI first produced a plausible playbook, but two details did not match the actual project. The follow-up prompt corrected the AI by referencing the custom module interface and Ansible variable scope.

Prompt excerpt

Revise the `site.yml` again because two details still do not match our actual Ansible project.

First change the `inventory_validate` argument name from `hostvars` to `firewall_hostvars`. Our custom Python module expects `firewall_hostvars`. If the argument is called `hostvars`, the module interface does not match our code.

Second, change the first play from `hosts: localhost` to `hosts: firewalls[0]` and put `delegate_to: localhost` plus `run_once: true` on the inventory validation task. This is needed because `firewall_objects` and `firewall_rules` are loaded from the `firewall` group variables. When the play runs only on `localhost`, those group variables are not available in the same way. Running the play in the context of the first firewall host gives Ansible access to the `firewall` group variables, while `delegate_to: localhost` still executes the custom validation locally.

Please output only the corrected final site.yml first. Then add a short explanation of these two corrections and why they are necessary in our project.

Manual review and reflection

This shows that the prompt was not asking AI to invent the architecture. The architecture and constraints were already known. The AI response had to be aligned with the actual custom module, the real group variable loading behavior, and the role order used by the project.

The correction also reflects an important Ansible concept from the course: variables are resolved in host and group context. A playbook can delegate execution to localhost while still evaluating variables from the selected managed host context.

Project result

This prompt chain affected ansible/playbooks/site.yml. The final deployment playbook validates the firewall inventory contract, then configures firewall nodes with the common, firewall, keepalived, and conntrackd roles, and then applies routing to backend servers and test clients.

Summary For The Common Team Document

Source note: selected and shortened from the longer Yedige Ansible AI documentation PDF. Complete version:

<https://docs.google.com/document/d/15P2ON8nWbpC8yPIZYH8-OSrFajq5AzmDLTRxhw9PafI/edit?usp=sharing>